# TA8: Finite Mixture Models: the Comeback!

06/11/2026

```
# Import modules
library(posterior)
library(bayesplot)
library(ggplot2)

# Import functions
library(cmdstanr)

# Create ./stan folder if does not exists
if (!dir.exists("./stan")) {
    dir.create("./stan")
}
```

In mixture model, we account for heterogeneity in the data by assuming that there is not “one single” data generating process, but actually $H$ of them.

Therefore, we expect to find $H$ different subpopulations (== clusters) in our data, and that each subpopulation is homogeneous: it is suitably modeled by a density, typically from a parametric family.

In its most general form, let $f_1(\cdot), \dots, f_H(\cdot)$ be $H$ probability density functions over a space $\mathbb{Y}, \mathcal{Y}$, $\mathbf{w} = (w_1, \dots, w_H)$ a vector in the $H$-simplex; a mixture model assumes the following likelihood

$$y_1, \dots, y_n \mid \mathbf{w}, \{f_h\}_{h=1}^H \overset{iid}{\sim} f^*(y) := \sum_{h=1}^H w_h f_h(y)$$

We refer to the $f_h$’s as the *components* of the mixture

The connection to clustering is made more explicit by introducing “cluster assignment” variables $c_i$, $i = 1, \dots, n$ such that

$$\mathbb{P}(c_i = h \mid \mathbf{w}) = w_h$$

Then, the mixture likelihood can be restated as the following hierarchical model

$$\begin{aligned} y_i \mid c_i = h, \{f_h\}_{h=1}^H &\overset{ind}{\sim} f_h \\ \mathbb{P}(c_i = h \mid \mathbf{w}) &= w_h \end{aligned}$$

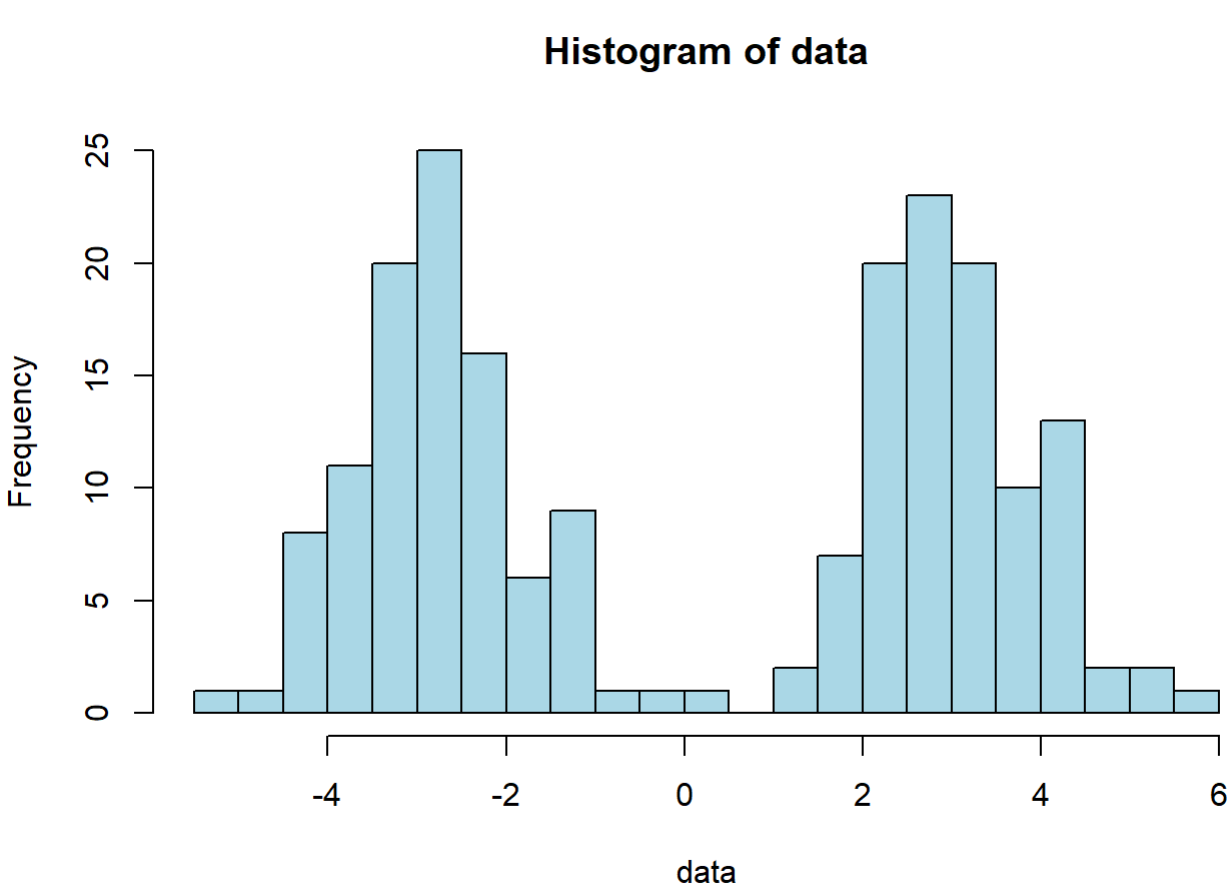
Usually, $\{f_j\}_{j=1}^K$ belong to the same parametric family (e.g., the normal distribution), and differ only for the specific values of the parameters (e.g., every component has a different mean/variance). But this might not always be the case.

# Gaussian Mixture Model

We consider a fictional dataset coming from the following distribution:

$$y_1, \dots, y_n \overset{ind}{\sim} \frac{1}{2}\mathcal{N}(-3.0, 1.0) + \frac{1}{2}\mathcal{N}(3.0, 1.0)$$

```
data <- c(rnorm(100,-3,1), rnorm(100,3,1))
hist(data, breaks = 20, col = 'lightblue')
```




And we assume the following Finite Mixture model for our data:

$$
\begin{aligned}
y_1, \dots, y_n \mid \mathbf{w}, \boldsymbol{\mu}, \boldsymbol{\sigma}^2 &\overset{ind}{\sim} \sum_{h=1}^{H} w_h \mathcal{N}(\mu_h, \sigma_h^2) \\
\mathbf{w} &\sim \text{Dirichlet}(\boldsymbol{\alpha}) \\
(\mu_h, \sigma_h) &\overset{iid}{\sim} \mathcal{N}(\mu_0, \sigma_0^2) \times \mathcal{HC}(0, \sigma_{max}), \qquad h = 1, \dots, H
\end{aligned}
$$

**How do we implement this model in `stan`?**

1. *From Stan Users' Guide (Section 7) (https://mc-stan.org/docs/stan-users-guide/latent-discrete.html):* > **Stan does not support sampling discrete parameters**. So it is not possible to directly translate BUGS or JAGS models with discrete parameters (i.e., discrete stochastic nodes). Nevertheless, it is possible to code many models that involve bounded discrete parameters by marginalizing out the discrete parameters.

This means that we can not use the model formulation with latent cluster allocation. We need to marginalize those $c_i$, so that we avoid the use of discrete parameters. But we already know which is the likelihood once the latent cluster allocation variables have been marginalized, don't we?

Ok, but we have a problem: such distribution is not implemented in stan!

2. *From Stan Reference Manual (Section 7.3) (https://mc-stan.org/docs/reference-manual/increment-log-prob.html)* > The basis of Stan's execution is the evaluation of a log probability function (specifically, a probability density function) for a given set of (real-valued) parameters; this function returns the log density of the posterior up to an additive constant. Data and transformed data are fixed before the log density is evaluated. The total log probability is initialized to zero. Next, any log Jacobian adjustments accrued by the variable constraints are added to the log density (the Jacobian adjustment may be skipped for optimization). Sampling and log probability increment statements may add to the log density in the model block. A log probability increment statement directly increments the log density with the value of an expression

This means that we actually have a way to implement a mixture model in `stan`! Let's see how can we do that.

```
gaussian_mix_uni <- "
data {
    // Data
    int N;
    int H;
    array[N] real y;

    // Prior parameters
    real alpha;
    array[3] real p0_params;
}

transformed data {
    real mu0 = p0_params[1];
    real sigma0 = p0_params[2];
    real sigma_max = p0_params[3];
}

parameters {
    array[H] real means;
    array[H] real<lower=0> std_devs;
    simplex[H] weights;
}

model {
    // Likelihood
    vector[H] contributions;
    for(i in 1:N) {
        for(h in 1:H) {
            contributions[h] = log(weights[h]) + normal_lpdf(y[i] | means[h], std_devs[h]);
        }
        target += log_sum_exp(contributions);
    }

    // Prior
    weights ~ dirichlet(rep_vector(alpha, H));
    means ~ normal(mu0, sigma0);
    std_devs ~ cauchy(0, sigma_max);
}
"

# Write stan model to file
stan_file <- "./stan/gaussian_mix_uni.stan"
writeLines(gaussian_mix_uni, stan_file)

# Compile stan model
gaussian_mix_uni <- cmdstan_model(stan_file)
```

```
gmm_data <- list(
    N = length(data),
    H = 4,
    y = data,
    alpha = 1.0,
    p0_params = c(0, 5, 1)
)

algo_params <- list(
    n_chains = 4,
    n_burnin = 1000,
    n_iter = 1000
)

gmm_fit <- gaussian_mix_uni$sample(
    data = gmm_data,
    chains = algo_params$n_chains,
    parallel_chains = algo_params$n_chains,
    iter_warmup = algo_params$n_burnin,
    iter_sampling = algo_params$n_iter,
    refresh = 500
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/gbelt/AppD
ata/Local/Temp/Rtmp8QvnZu/model-68845f8c4639.stan', line 30, column 12 to column 91)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

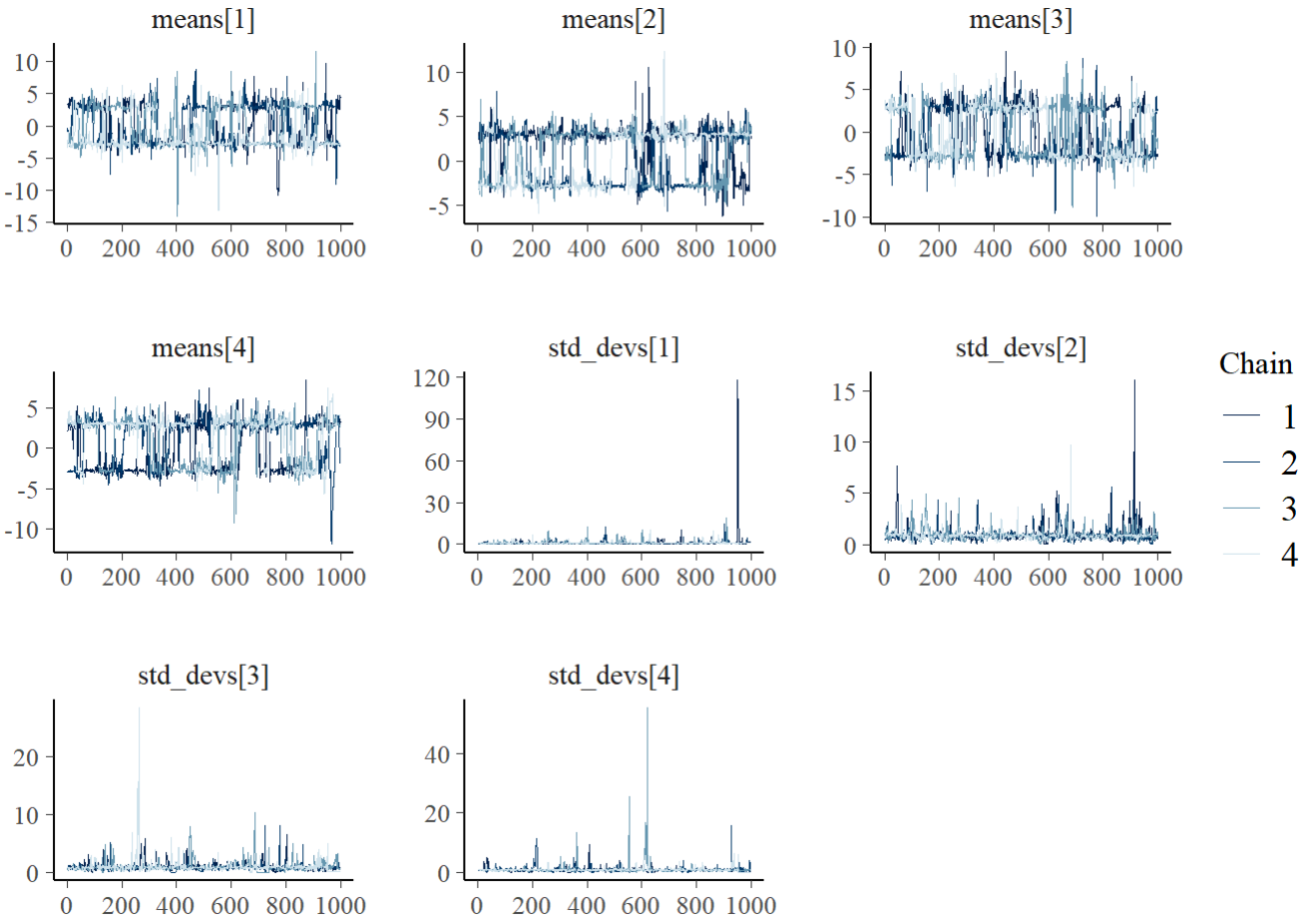
```
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

```
## Chain 1
```

```
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 23.2 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 24.9 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 25.2 seconds.
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 26.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 24.9 seconds.
## Total execution time: 26.4 seconds.
```

```
chains <- posterior::as_draws(gmm_fit)
```

```
mcmc_trace(gmm_fit$draws(c("means", "std_devs")))
```




Looking at the chains of the cluster specific parameters is misleading, and we know why! What may happen is that either the posterior distribution of those parameters is multi-modal (thus meaning taht the model is not identifiable) or there is no agreement between parallel chains.

**Take Home Message**: you can assess convergence of the MCMC for a mixture model only looking at the common parameters, i.e., those that do not depend on $H$!

# Posterior Predictive Distribution

The type of checks that make sense with mixture models are predictive checks. Hence, we can compute the posterior predictive density and compare it with the empirical density histogram of the data.

Given a new value $y^*$, we need to compute:

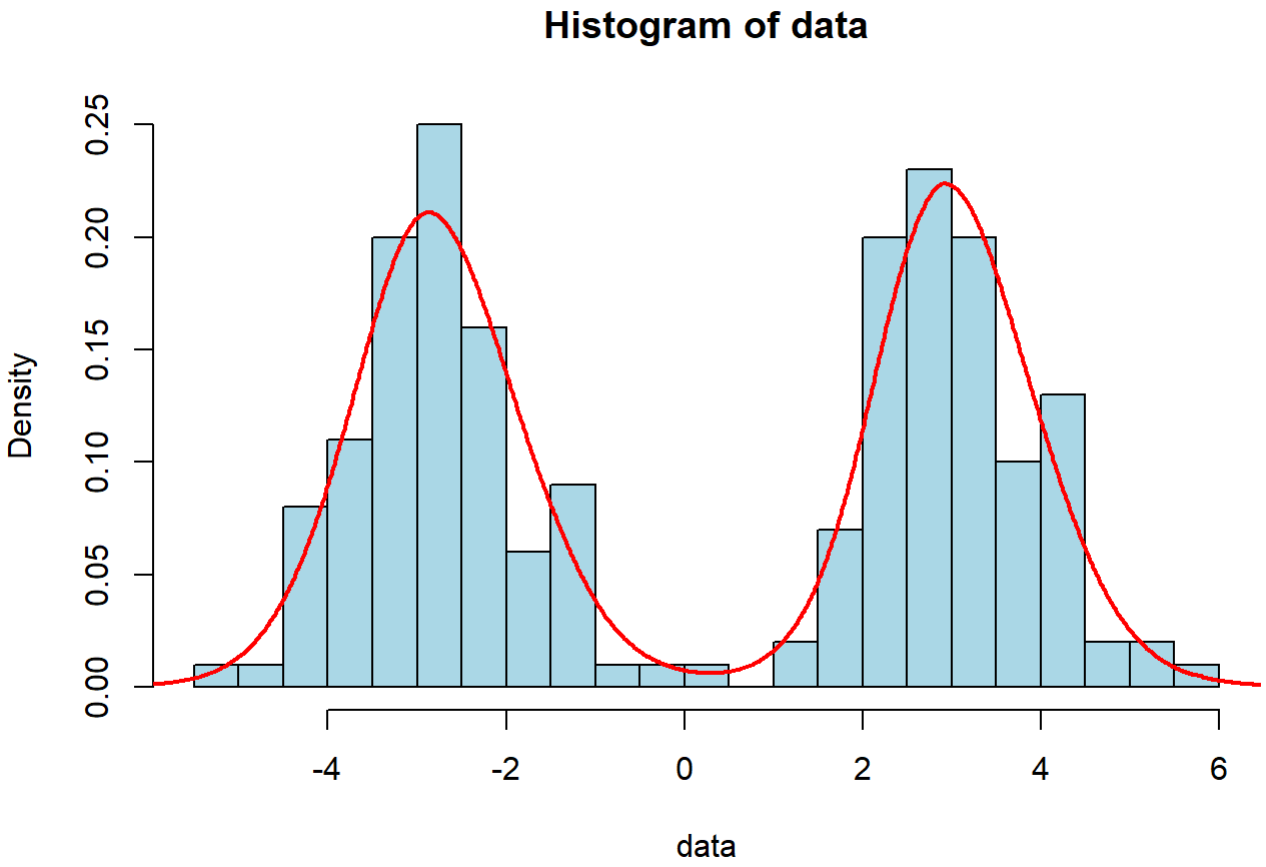
$$\begin{aligned}\mathcal{L}(y^* \mid \mathbf{y}) &= \int \mathcal{L}(y^* \mid \mathbf{w}, \boldsymbol{\mu}, \boldsymbol{\sigma}^2)\, \pi(\mathbf{w} \mid \mathbf{y})\, \pi(\boldsymbol{\mu} \mid \mathbf{y})\, \pi(\boldsymbol{\sigma}^2 \mid \mathbf{y})\, \mathrm{d}\mathbf{w}\, \mathrm{d}\boldsymbol{\mu}\, \mathrm{d}\boldsymbol{\sigma}^2, \\ &\approx \frac{1}{M} \sum_{j=1}^{M} \sum_{h=1}^{H} w_h^{(j)} \mathcal{N}\left(y^* \mid \mu_h^{(j)}, \sigma_h^{2(j)}\right),\end{aligned}$$

where $\{\mathbf{w}^{(j)}\}_{j=1}^{M}$, $\{\boldsymbol{\mu}^{(j)}\}_{j=1}^{M}$ and $\{\boldsymbol{\sigma}^{2(j)}\}_{j=1}^{M}$ are the corresponding MCMC chains we retrieve from `stan`. This holds for a single value $y^* \in \mathbb{R}$. In order to retrieve a density, we can compute the same quantity over a grid of points.

```
predictive_gaussian_mix_uni <- function(chains, x_grid) {
    weights <- matrix(posterior::subset_draws(chains, variable = "weights"), nrow = 4000, ncol = 4, byr
ow = F)
    means <- matrix(posterior::subset_draws(chains, variable = "means"), ncol = 200, nrow = 4000, byrow
= F)
    std_devs <- matrix(posterior::subset_draws(chains, variable = "std_devs"), ncol = 200, nrow = 4000,
byrow = F)
    num_components <- ncol(weights)
    out <- matrix(0, nrow = nrow(weights), ncol = length(x_grid))
    for (i in 1:nrow(out)) {
        eval_dens <- rep(0, length(x_grid))
        for (h in 1:num_components) {
            eval_dens <- eval_dens + weights[i, h] * dnorm(x_grid, mean = means[i, h], sd = std_devs[i,
h])
        }
        out[i, ] <- eval_dens
    }
    return(colMeans(out))
}
```

```
x <- seq(-8, 8, length.out = 1000)
post_dens <- predictive_gaussian_mix_uni(chains, x)

hist(data, breaks = 20, probability = T, col = 'lightblue')
lines(x, post_dens, col = 'red', lwd = 2)
```




### What about clustering?

We already know that Mixture Models are aminly used for clustering. Clustering is obtained via the latent cluster allocation variables $\{c_i\}_{i=1}^N$ that are naturally introduced if we need to implement a Gibbs Sampler.

We can use a trick to retrieve the latent cluster allocation *after* the sampling. We can do it in `stan`, in the `generated quantities` block, or in Python. What e need to do is to sample from its full conditional distribution for every state of the chain, like we would do in a Gibbs sampler. Hence:

$$\begin{aligned} c_i = h \mid rest &\propto \mathcal{L}(y_i \mid c_i = h, \mathbf{w}, \boldsymbol{\mu}, \boldsymbol{\sigma}^2)\mathbb{P}(c_i = h \mid \mathbf{w}), \\ &= w_h \mathcal{N}(y_i \mid \mu_h, \sigma_h^2), \end{aligned}$$

for $i = 1, \ldots, N$ and for $h = 1, \ldots, H$.

```
gaussian_mix_uni_clus <- "
data {
    // Data
    int N;
    int H;
    array[N] real y;

    // Prior parameters
    real alpha;
    array[3] real p0_params;
}

transformed data {
    real mu0 = p0_params[1];
    real sigma0 = p0_params[2];
    real sigma_max = p0_params[3];
}

parameters {
    array[H] real means;
    array[H] real<lower=0> std_devs;
    simplex[H] weights;
}

model {
    // Likelihood
    vector[H] contributions;
    for(i in 1:N) {
        for(h in 1:H) {
            contributions[h] = log(weights[h]) + normal_lpdf(y[i] | means[h], std_devs[h]);
        }
        target += log_sum_exp(contributions);
    }

    // Prior
    weights ~ dirichlet(rep_vector(alpha, H));
    means ~ normal(mu0, sigma0);
    std_devs ~ cauchy(0, sigma_max);
}

generated quantities {
  array[N] int cluster_allocs;
  for(i in 1:N){
    vector[H] log_probs = log(weights);
    for(h in 1:H){
      log_probs[h] += normal_lpdf(y[i] | means[h], std_devs[h]);
    }
    cluster_allocs[i] = categorical_rng(softmax(log_probs));
  }
}
"

# Write stan model to file
stan_file <- "./stan/gaussian_mix_uni_clus.stan"
writeLines(gaussian_mix_uni_clus, stan_file)

# Compile stan model
gaussian_mix_uni_clus <- cmdstan_model(stan_file)
```

```
gmm_data <- list(
    N = length(data),
    H = 4,
    y = data,
    alpha = 0.1,
    p0_params = c(0, 5, 5)
)

algo_params <- list(
    n_chains = 4,
    n_burnin = 1000,
    n_iter = 1000
    )

gmm_clus_fit <- gaussian_mix_uni_clus$sample(
    data = gmm_data,
    chains = algo_params$n_chains,
    parallel_chains = algo_params$n_chains,
    iter_warmup = algo_params$n_burnin,
    iter_sampling = algo_params$n_iter,
    refresh = 500,
    adapt_delta = 0.95,
    max_treedepth = 15
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/gbelt/AppD
ata/Local/Temp/Rtmp8QvnZu/model-68842259c7f.stan', line 30, column 12 to column 91)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

```
## Chain 1
```

```
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 132.1 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 147.1 seconds.
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 190.8 seconds.
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 197.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 166.8 seconds.
## Total execution time: 197.2 seconds.
```

```
chains <- posterior::as_draws(gmm_clus_fit)
```

# Point estimates with Mixture Models

We know that mixture models are non-identifiable, as we can see from the MCMC chains we retrieve from `stan`. There are several ways to solve this non-identifiability issue and retrieve point estimates for the clustering and for the cluster-specific parameters. What is done, usually, is to find a point estimate for the cluster and then retrieve the cluster-specific variables once the cluster is known.

To find a point estimate for the cluster, we will follow a decision-theoretic approach. The idea is to use a loss function and find the partition that minimizes such loss function. One of the most known loss function for clustering is the Binder's loss, defined as:

$$\sum_{i=1}^{n}\sum_{j=1}^{n}(\mathbb{I}[c_i = c_j] - p_{ij})^2$$

where $p_{ij}$ is the posterior probability that observations $i$ and $j$ belong to the same cluster, i.e., $\mathbb{P}(c_i = c_j \mid \mathbf{y})$.

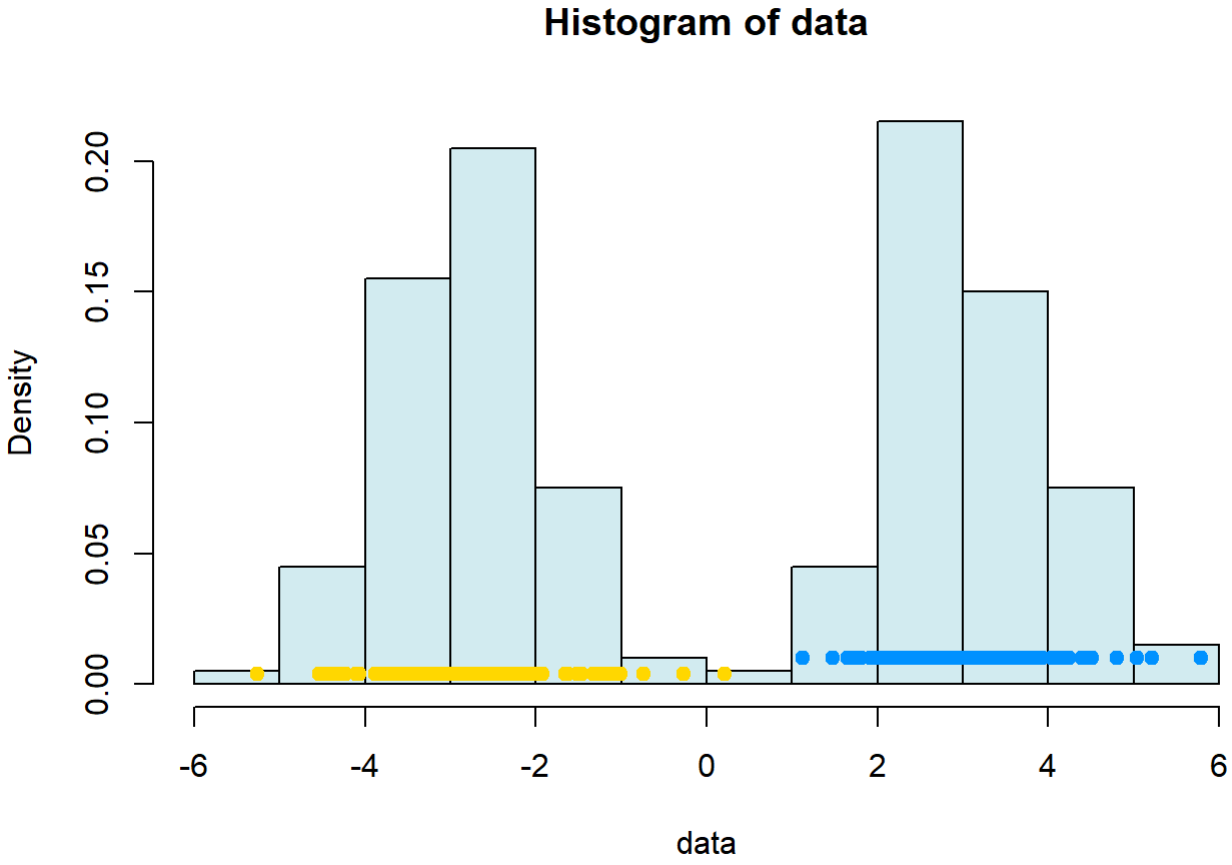
```
get_psm <- function(clus_alloc_chain) {
    c_chain <- as.matrix(clus_alloc_chain)
    n <- ncol(c_chain)
    out <- matrix(0, nrow = n, ncol = n)
    for (i in 1:n) {
        for (j in 1:(i - 1)) {
            out[i, j] <- mean(c_chain[, i] == c_chain[, j])
        }
    }
    return(out + t(out) + diag(n))
}

minbinder_sample <- function(clus_alloc_chain, psm) {
    c_chain <- as.matrix(clus_alloc_chain)
    losses <- numeric(nrow(c_chain))
    n_obs <- ncol(c_chain)
    for (s in nrow(c_chain)) {
    same_cluster <- outer(c_chain[s, ], c_chain[s, ], "==")
    losses[s] <- sum((same_cluster - psm)^2)
    }

    best_iter <- which.min(losses)
    return(c_chain[best_iter, ])
}
```

```
clust_allocs_chain <- matrix(posterior::subset_draws(chains, variable = "cluster_allocs"), ncol = 200,
nrow = 4000, byrow = F)
psm <- get_psm(clust_allocs_chain)
best_clus <- minbinder_sample(clust_allocs_chain, psm)
```

```
cols <- rainbow(7)
hist(data, probability = TRUE, col = adjustcolor("lightblue", alpha.f = 0.5))
for (h in unique(best_clus)) {
    currd <- data[best_clus == h]
    points(currd, rep(0.002 * (h + 1), length(currd)), col = cols[h + 1], pch = 19)
}
```




Then, given the point estimate of the clustering $\mathbf{c}^*$, we can estimate the parameters in each cluster. Let $c_h^*$ denote the index-set of the $h$–th cluster (e.g., $c_1^* = \{1, 3, 10\}$, $c_2^* = \{0, 2, 4, 5, 6\}$ ...) Then, for every cluster-specific parameter $\theta_h$, we can estimate it as:

$$\hat{\theta}_h = \frac{1}{M} \sum_{j=1}^{M} \frac{1}{\#c_h^*} \sum_{i \in c_h^*} \theta_{c_i^{(j)}}^{(j)}$$

where $\mu_\ell^{(j)}$ is the mean of the $\ell$–th cluster at the $j$–th MCMC iteration. Similarly for the variance $\hat{\sigma}_h^2$.

```
unique_vals_given_clus <- function(chains, best_clus) {
    c_allocs <- matrix(posterior::subset_draws(chains, variable = "cluster_allocs"), ncol = 200, nrow = 4000, byrow = F)
    means <- matrix(posterior::subset_draws(chains, variable = "means"), ncol = 4, nrow = 4000, byrow = F)
    std_devs <- matrix(posterior::subset_draws(chains, variable = "std_devs"), ncol = 4, nrow = 4000, byrow = F)
    out <- list()
    uniq_h <- unique(best_clus)
    for (h in uniq_h) {
        # Find indexes of data in cluster h and corresponding cluster allocations
        data_idx <- which(best_clus == h)
        uniq_vals_idx <- c_allocs[, data_idx]
        # Prepare and fill buffers
        means_by_iter <- matrix(0, nrow = nrow(c_allocs), ncol = length(data_idx))
        std_devs_by_iter <- matrix(0, nrow = nrow(c_allocs), ncol = length(data_idx))
        for (i in 1:nrow(c_allocs)) {
            col_indices <- uniq_vals_idx[i, ]
            means_by_iter[i, ] <- means[i, col_indices]
            std_devs_by_iter[i, ] <- std_devs[i, col_indices]
        }
        # Compute point estimate of cluster specific parameters
        avg_mean_by_iter <- rowMeans(means_by_iter)
        avg_stdev_by_iter <- rowMeans(std_devs_by_iter)
        muhat <- mean(avg_mean_by_iter)
        sighat <- mean(avg_stdev_by_iter)
        out[[length(out) + 1]] <- c(muhat, sighat)
    }
    return(out)
}

clus_vals <-  unique_vals_given_clus(chains, best_clus)
clus_vals
```

```
## [[1]]
## [1] -2.7371223  0.9681927
##
## [[2]]
## [1] 3.0530728 0.9268195
```

# Homework - The (well known) Edgar Anderson's iris dataset

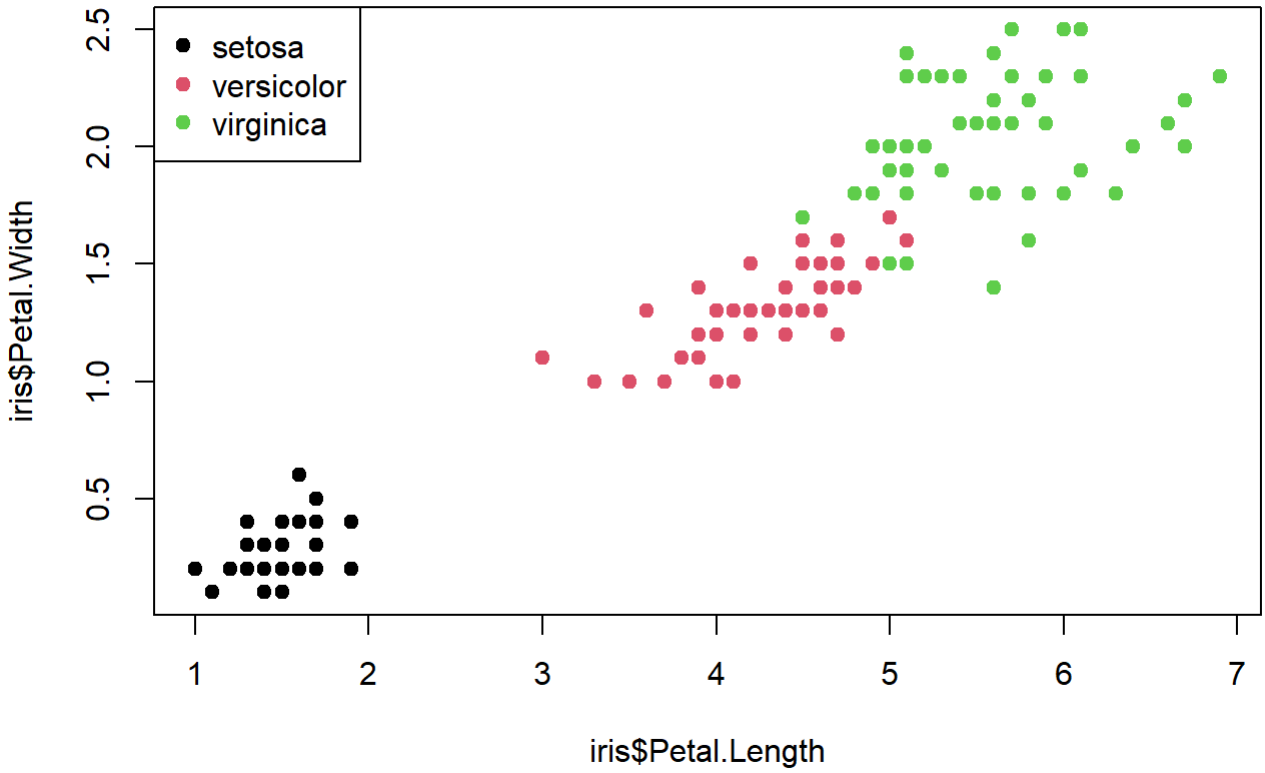
The iris dataset gives the measurements in centimeters of the variables sepal length and width and petal length and width, respectively, for 50 flowers from each of 3 species of iris. The species are Iris *setosa*, *versicolor*, and *virginica*.

`iris` is a data frame with 150 cases (rows) and 5 variables (columns) named `Sepal.Length`, `Sepal.Width`, `Petal.Length`, `Petal.Width`, and `Species`.

```
data("iris")
```

We want to cluster iris species according to the petal length and width. The `Species` column contains the true cluster, hence we also have a ground truth for comparison. Let us see the data grouped by their true species:

```
plot(iris$Petal.Length, iris$Petal.Width, col = as.integer(iris$Species), pch = 19)
legend("topleft", legend = levels(iris$Species), col = 1:3, pch = 19)
```




The *setosa* group is well separated from the other two species. *versicolor* and *virginica* are closer but with few superposition. In general the clusters seems to be well separated.

# A Bivariate Mixture model

If the main inference goal is clustering, we are assuming data are heterogeneous and comes from different populations. We also assume that the whole dataset is heterogeneous but within each group, data are homogeneous. Clustering is nothing more than finding a data partition such that the above assumptions holds.

The natural way of modeling this type of data in Bayesian Statistics is via Mixture models. In mixture model, we account for heterogeneity in the data by assuming that there is not “one single” data generating process, but actually $H$ of them. Therefore, we expect to find $H$ different subpopulations (== clusters) in our data.

In our case, we assume that our data (i.e. the petal length and width of iris) comes from a mixture of **bivariate Gaussian distributions**:

$$\mathbf{y}_1, \dots, \mathbf{y}_n \overset{\text{iid}}{\sim} \sum_{h=1}^{H} \mathbf{w}_h \, \mathcal{N}_2 \left( \boldsymbol{\mu}_h, \Sigma_h \right)$$

**Questions**: 1. Which type of prior would you choose for $\left( \boldsymbol{\mu}_h, \Sigma_h \right)$? How would you fix the hypterparameters? Try to implement it in `stan` ! 2. Implement a function that samples from the posterior predictive distribution of our model. 3. Since we have the true classification, compare the best cluster estimate you obtain (by minimizing the binder loss function) with the true classification. 4. Provide an estimate for the cluster-specific parameters $\left( \boldsymbol{\mu}_h, \Sigma_h \right)$ for $h = 1, \dots, H$.

```
gaussian_mix_iris <- "
data { }

transformed data { }

parameters { }

transformed parameters { }

model { }

generated quantities { }
"

# Write stan model to file
stan_file <- "./stan/gaussian_mix_iris.stan"
writeLines(gaussian_mix_iris, stan_file)

# Compile stan model
gaussian_mix_iris <- cmdstan_model(stan_file)
```

```
# Input data
iris_data <- list(
    N = nrow(iris),
    H = 3,
    y = as.matrix(iris[, c("Petal.Length", "Petal.Width")])
)

# Sample
gaussian_mix_iris_fit <- gaussian_mix_iris$sample(
    data = iris_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000
)

# Convert to arviz data type
chains <- posterior::as_draws(gaussian_mix_iris_fit)
```

```
clust_allocs_chain <- matrix(posterior::subset_draws(chains, variable = "cluster_allocs"), ncol = nrow
(iris), nrow = 4000, byrow = F)
psm <- get_psm(clust_allocs_chain)
best_clus <- minbinder_sample(clust_allocs_chain, psm)
```

```r
# Add best cluster estimate to dataset
data <- iris[, c("Petal.Length", "Petal.Width")]
data$Clus <- best_clus

# Compare true and estimated cluster
par(mfrow = c(1, 2))

# Plot - true clusters
plot(iris$Petal.Length, iris$Petal.Width, col = as.integer(iris$Species), pch = 19, main = "True", xlab
= "Petal.Length", ylab = "Petal.Width")
legend("topleft", legend = levels(iris$Species), col = 1:3, pch = 19)

# Plot - estimated clusters
unique_clus <- unique(data$Clus)
plot(data$Petal.Length, data$Petal.Width, col = data$Clus + 1, pch = 19, main = "Estimated", xlab = "Pe
tal.Length", ylab = "Petal.Width")
legend("topleft", legend = unique_clus, col = unique_clus + 1, pch = 19)

par(mfrow = c(1, 1))
```